推理服务挑战赛 — 初赛开发文档

一、概述

本文档面向初赛（预赛）阶段的参赛选手，说明代码运行环境、交互协议、配置格式和计分细则。

初赛形式：选手通过邮件提交代码包，平台在统一测试环境中运行选手代码，输入动态任务流，记录推理结果与延迟表现，按总分排名。

评测范围：评测任务涵盖多个领域，包括但不限于：数学推理、常识问答、语言理解、知识推理、语言建模等。具体题目在比赛各阶段可能有所调整。

二、运行环境

2.1 硬件配置

资源	规格
GPU	*待确定，比赛前公布*
CPU	*待确定，比赛前公布*
内存	*待确定，比赛前公布*
网络	选手服务与评测平台在同一局域网内，RTT < 1ms

2.2 软件环境

组件	版本
操作系统	Ubuntu 22.04
Python	3.12
CUDA	12.8
模型权重	平台提供，挂载在固定路径（比赛前公布）

2.3 选手代码要求

选手提交一个代码包（`.tar.gz`），需包含：

Code block

```
submission/
├── setup.sh            # 环境安装脚本（可选）
├── run.sh              # 启动脚本（必须）
├── requirements.txt    # Python 依赖（可选）
└── ...                 # 选手代码
```

`setup.sh` 规范：

- 若提供 `setup.sh`，平台在首次运行时执行 `bash setup.sh` 完成环境安装
- 允许创建虚拟环境、安装 pip 依赖、编译自定义算子等
- 安装完成后的环境会被缓存，后续运行不再重复执行
- 系统预装 `python3.12`、`pip`、`uv`，可直接使用

`setup.sh` 示例：

Code block

```bash
#!/bin/bash
python3.12 -m venv /tmp/contestant_env
source /tmp/contestant_env/bin/activate
pip install -r requirements.txt
```

`run.sh` 规范：

- 平台执行 `bash run.sh` 启动选手服务
- 若使用了虚拟环境，需在 `run.sh` 中自行激活
- 选手服务必须在 **60 秒内** 启动完成，并在指定端口上提供 HTTP 服务
- 选手服务的端口通过环境变量 `CONTESTANT_PORT` 传入（默认 9000）
- 模型权重路径通过环境变量 `MODEL_PATH` 传入
- 配置文件路径通过环境变量 `CONFIG_PATH` 传入

环境变量汇总：

变量	说明	示例
`CONTESTANT_PORT`	选手 HTTP 服务监听端口	`9000`
`MODEL_PATH`	模型权重目录	`/mnt/model/Qwen3-32B`
`CONFIG_PATH`	比赛配置文件路径	`/mnt/config/contest.json`
`PLATFORM_URL`	评测平台地址	`http://10.0.0.1:8003`

三、比赛配置文件

平台通过 `CONFIG_PATH` 提供的 JSON 配置文件，包含本轮比赛的关键参数：

Code block

```
{
  "platform_url": "http://10.0.0.1:8003",
  "model_name": "Qwen3-32B",
  "model_path": "/mnt/model/Qwen3-32B",
  "contestant_port": 9000,
  "duration_s": 3600,

```

```
  "sla_levels": {
    "Bronze":   {"ttft_avg": 10.0},
    "Silver":   {"ttft_avg": 8.0},
    "Gold":     {"ttft_avg": 6.0},
    "Platinum": {"ttft_avg": 4.0},
    "Diamond":  {"ttft_avg": 2.0},
    "Stellar":  {"ttft_avg": 1.5},
    "Glorious": {"ttft_avg": 0.8},
    "Supreme":  {"ttft_avg": 0.5}
  },

  "sampling_params": {
    "Deterministic":  {"temperature": 0.0, "top_p": 1.0,  "top_k": 1, "repetition_penalty": 1.0,  "frequency_penalty": 0.0, "presence_penalty": 0.0},
    "Normal":         {"temperature": 0.1, "top_p": 0.9,  "top_k": 50, "repetition_penalty": 1.1,  "frequency_penalty": 0.2, "presence_penalty": 0.2},
    "HighEntropy":    {"temperature": 0.1, "top_p": 0.95, "top_k": 100, "repetition_penalty": 1.05, "frequency_penalty": 0.0, "presence_penalty": 0.0},
    "ExtremePenalty":  {"temperature": 0.1, "top_p": 0.9,  "top_k": 20, "repetition_penalty": 1.8,  "frequency_penalty": 1.2, "presence_penalty": 1.2}
  }
}
```

注：ttft是从bid完成开始到收到response的时间

四、交互协议

4.1 整体流程

Code block

```
选手服务启动
    ↓
POST /register          注册身份
    ↓
┌→ POST /query           拉取一道任务概要
│     ↓
│  POST /ask             接受任务（需匹配指定 SLA）
│     ↓
│  收到完整任务（含 prompt、采样参数等）
│     ↓
```

```
│  选手推理
│      ↓
│  POST /submit          提交结果
│      ↓
└──────────────────────────      循环
```

4.2 注册

请求： `POST {PLATFORM_URL}/register`

Code block

```
{
  "name": "team_alpha",
  "token": "your_secret_token"
}
```

响应：

Code block

```
{"status": "ok"}
```

`token` 是选手的唯一标识，由平台预分配。所有后续请求都需要携带 `token`。

4.3 查询任务

请求： `POST {PLATFORM_URL}/query`

Code block

```
{
  "token": "your_secret_token"
}
```

响应（200）：任务概要

```
{
  "task_id": 2000000042,
  "target_sla": "Gold",
  "target_reward": 1.2,
  "max_winners": 1,
  "eval_task_name": "task_001",
  "eval_request_type": "loglikelihood",
  "eval_sampling_param": "",
  "eval_timeout_s": 600
}
```

响应（404）：当前无可用任务，稍后重试。

关键字段说明：

字段	说明
`task_id`	任务唯一 ID
`target_sla`	本题的 SLA 等级（初赛固定，不可更改）
`target_reward`	满分收益
`eval_request_type`	推理类型：`generate_until` / `loglikelihood` / `loglikelihood_rolling`
`eval_sampling_param`	采样参数名（仅 generate_until 有值）
`eval_timeout_s`	绝对超时（秒）

4.4 接受任务

请求：`POST {PLATFORM_URL}/ask`

Code block

```
{
  "token": "your_secret_token",
  "task_id": 2000000042,
  "sla": "Gold"
}
```

初赛规则： `sla` 字段必须与 `target_sla` 完全匹配，否则被拒绝。

响应（接受）：

Code block

```
{
  "status": "accepted",
  "task": {
    "overview": { ... },
    "messages": [
      {
        "ID": 0,
        "prompt": "Question: Which of the following...",
        "eval_req_id": "w0_a1b2c3d4",
        "eval_request_type": "loglikelihood",
        "eval_gen_kwargs": null,
        "eval_continuation": "choice A text"
      },
      {
        "ID": 1,
        "prompt": "Question: Which of the following...",
        "eval_req_id": "w0_b5c6d7e8",
        "eval_request_type": "loglikelihood",
        "eval_continuation": "choice B text"
      }
    ]
  }
}
```

响应（拒绝）：

Code block

```
{"status": "rejected", "reason": "SLA must match target"}
```

或

```
Code block
1   {"status": "closed"}
```

4.5 Message 字段说明

每个 message 代表一条推理请求。根据 `eval_request_type` 的不同，选手需要进行不同的推理：

generate_until

```
Code block
1   {
2     "ID": 0,
3     "prompt": "Question: Natalia sold clips...\nAnswer:",
4     "eval_request_type": "generate_until",
5     "eval_gen_kwargs": {
6       "until": ["\n\n"],
7       "max_gen_toks": 256,
8       "temperature": 0.0,
9       "top_p": 1.0
10    },
11    "eval_continuation": null
12  }
```

选手需要：

1. 使用 `prompt` 作为输入
2. 按 `eval_gen_kwargs` 中的采样参数进行生成
3. 遇到 `until` 中的停止符或达到 `max_gen_toks` 时停止
4. 将生成的文本填入 `response` 字段

loglikelihood

Code block

```
{
  "ID": 0,
  "prompt": "Context text...",
  "eval_request_type": "loglikelihood",
  "eval_continuation": "choice A",
  "eval_gen_kwargs": null
}
```

选手需要：

1. 计算 `P(eval_continuation | prompt)` 的对数概率
2. 将 logprob 值填入 `accuracy` 字段

注意：一道多选题会拆分成多个 message（每个候选答案一条）。平台根据各候选的 logprob 选择 argmax 来判断选手的回答是否正确。

loglikelihood_rolling

Code block

```
{
  "ID": 0,
  "prompt": "A long document text...",
  "eval_request_type": "loglikelihood_rolling",
  "eval_continuation": null,
  "eval_gen_kwargs": null
}
```

选手需要：

1. 计算整段 `prompt` 文本的 rolling log-likelihood
2. 将总 logprob 值填入 `accuracy` 字段

4.6 提交结果

请求：`POST {PLATFORM_URL}/submit`

Code block

```
{
  "user": {
```

```
      "name": "team_alpha",
      "token": "your_secret_token"
    },
    "msg": {
      "overview": { ... },
      "messages": [
        {
          "ID": 0,
          "prompt": "...",
          "response": "生成的文本",
          "accuracy": null,
          "eval_req_id": "w0_a1b2c3d4",
          "eval_request_type": "generate_until",
          ...
        },
        {
          "ID": 1,
          "prompt": "...",
          "response": null,
          "accuracy": -2.35,
          "eval_req_id": "w0_b5c6d7e8",
          "eval_request_type": "loglikelihood",
          ...
        }
      ]
    }
  }
```

结果填充规则：

request_type	填充字段	类型	说明
`generate_until`	`response`	string	生成的文本
`loglikelihood`	`accuracy`	float	对数概率
`loglikelihood_rolling`	`accuracy`	float	总对数概率

响应：

```
{"status": "ok"}
```

五、计分细则

5.1 单题收益

在 SLA 时限内完成的任务：

Code block

```
R_i = w_task × w_sla × w_sp × C_i
```

其中：

- `w_task`：任务基础权重，正比于推理计算量（`generate_until` 类最高，`loglikelihood` 类按候选数加权）
- `w_sla`：SLA 难度乘数（Bronze = 1.0，逐级递增至 Supreme = 2.5）
- `w_sp`：采样参数难度乘数（仅 `generate_until` 有效，Deterministic = 1.0，ExtremePenalty = 1.3）
- `C_i`：正确性得分（0 ~ 1.0）

5.2 正确性评判

- **`generate_until`**：选手生成结果与参考模型对比，按标准指标计算 `C_i = absolute_score / reference_score`（不超过1）
- **`loglikelihood`**：选手的 argmax 选择与参考模型是否一致（一致 = 1.0，不一致 = 0.0）
- **`loglikelihood_rolling`**：困惑度是否在参考值合理范围内

5.3 超时规则

状态	条件	得分
SLA 内完成	端到端延迟满足 SLA 要求	R_i（正常得分）
SLA 超时完成	超过 SLA 但在 600 秒内提交	0（不扣分）
未完成	600 秒内未提交	-2 × w_task × w_sla × w_sp

六、客户端示例

6.1 最小可运行示例（Python）

Code block

```python
import httpx
import time

PLATFORM_URL = "http://10.0.0.1:8003"
TOKEN = "your_secret_token"
INFERENCE_URL = "http://localhost:9000"  # 选手自己的推理服务

client = httpx.Client(timeout=60.0)

# 1. 注册
client.post(f"{PLATFORM_URL}/register", json={"name": "my_team", "token": TOKEN})

while True:
    # 2. 查询
    resp = client.post(f"{PLATFORM_URL}/query", json={"token": TOKEN})
    if resp.status_code != 200:
        time.sleep(0.5)
        continue

    overview = resp.json()
    task_id = overview["task_id"]
    target_sla = overview["target_sla"]
```

```python

    # 3. 接受（初赛必须匹配 target_sla）
    resp = client.post(f"{PLATFORM_URL}/ask", json={
        "token": TOKEN, "task_id": task_id, "sla": target_sla
    })
    result = resp.json()
    if result["status"] != "accepted":
        continue

    task = result["task"]

    # 4. 推理
    for msg in task["messages"]:
        rt = msg.get("eval_request_type", "loglikelihood")
        prompt = msg["prompt"]

        if rt == "generate_until":
            gen_kwargs = msg.get("eval_gen_kwargs", {})
            # 调用选手推理服务的 /v1/completions
            inf_resp = client.post(f"{INFERENCE_URL}/v1/completions", json={
                "model": "Qwen3-32B",
                "prompt": prompt,
                "max_tokens": gen_kwargs.get("max_gen_toks", 256),
                "temperature": gen_kwargs.get("temperature", 0.0),
                "stop": gen_kwargs.get("until", []),
            })
            msg["response"] = inf_resp.json()["choices"][0]["text"]

        elif rt == "loglikelihood":
            continuation = msg.get("eval_continuation", "")
            # 计算 P(continuation | prompt)
            msg["accuracy"] = compute_logprob(prompt, continuation)

        elif rt == "loglikelihood_rolling":
            # 计算整段文本的 rolling log-likelihood
            msg["accuracy"] = compute_rolling_logprob(prompt)

    # 5. 提交
    client.post(f"{PLATFORM_URL}/submit", json={
        "user": {"name": "my_team", "token": TOKEN},
        "msg": task,
    })
```

七、提交清单

项目	必须	说明
`run.sh`	✅	启动脚本，60 秒内完成启动
`setup.sh`	建议	环境安装脚本（创建虚拟环境、安装依赖等）
选手代码	✅	推理服务 + 调度客户端
`requirements.txt`	建议	额外 Python 依赖
技术报告	建议	描述系统架构和优化策略（≤ 2 页）

八、常见问题

Q：loglikelihood 任务的多个 message 是什么关系？

A：一道多选题被拆成多个 message，每个 message 对应一个候选答案。例如 4 选 1 的题会生成 4 条 message，它们共享相同的 `prompt`（题目上下文），但 `eval_continuation` 不同（各个选项文本）。选手需要分别计算每个候选的 logprob，平台根据 argmax 判断正误。

Q：SLA 超时后还能提交吗？

A：可以。SLA 超时后提交结果只是不得分（0 分），但不会扣分。只有 600 秒绝对超时后仍未提交才会扣分。

Q：同一个 prompt 会重复出现吗？

A：题目会随机从题库中抽取，有可能出现相同 prompt（但推理要求不同）。选手可以利用推理缓存（KV Cache / Prefix Caching）来加速重复前缀的推理。

Q：generate_until 的采样参数必须遵守吗？

A：是的。`eval_gen_kwargs` 中的 temperature、top_p、top_k 等参数是评测条件的一部分。不使用指定参数可能导致生成结果与参考答案不匹配，降低正确性得分。

Q：可以选择性地只做某些类型的任务吗？

A：可以。选手可以通过 `/query` 查看任务概要后决定是否 `/ask`。但请注意，不做的任务不会计分——既不得分也不扣分（除非已 ask 但不 submit）。